

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 4

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Part 1: Fix sleep for the current xv6 **3**

2 Part 2: bash versus the xv6 shell **3**

3 Part 3: Who expands *? A program that prints its argv **7**

4 Part 4: Read the shell **14**

Part 1: Fix sleep for the current xv6

cd ~/xv6-riscv && grep -n "pause\|sleep" user/user.h – moves into the xv6 source tree and searches the file `user/user.h`, which lists the system call prototypes available to user programs, for the names `pause` and `sleep`. In August 2025 the authors of xv6 renamed the system call `sleep()` to `pause()`, so the result of this command shows which name the current tree uses. As the tree used in this work declares `pause`, the call in `user/sleep.c` had to be changed.

nano user/sleep.c – the program from Laboratory Work No. 2 was edited so that it calls `pause()` instead of `sleep()`. The program checks that exactly one argument was given, prints the usage message on the standard error (file descriptor 2) otherwise, converts the argument to an integer with `atoi` and asks the kernel to pause the process for that number of clock ticks:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1])); // was sleep() before Aug 2025
  exit(0);
}
```

Makefile – UPROGS – the line `$U/_sleep\` must be present in the variable `UPROGS`, otherwise the program is not compiled and is not copied into the xv6 file system. Every such line starts with a TAB and ends with a backslash, and nothing may follow the backslash.

make qemu – compiles the kernel and all the user programs, builds the file system image and starts xv6 in the QEMU emulator. After booting, the commands `sleep 20` (the prompt returns after about 20 ticks) and `sleep` without argument (which prints `usage: sleep <ticks>`) verify that the program works. QEMU is closed with Ctrl-A followed by X.

Part 2: bash versus the xv6 shell

Questions:

- List the features both shells have, and the ones only bash has.
- In xv6, ls stops working after cd d. Why? Find a way to run ls from inside d anyway. (Hint: where do the xv6 programs live?)

- Which missing feature would you miss most in daily work, and which would be easiest to add?

Answers:

- Both shells have input and output redirection (`>` and `<`), pipes (`|`), grouping of commands with parentheses, sequences of commands separated by `;`, background execution with `&` and the built-in command `cd`. Only bash has wildcard expansion (`*`), variables (`$HOME`), quote removal (single and double quotes), a search path (`PATH`) for programs, and an interactive line editor with command history (arrow up, Ctrl+R) and Tab completion.
- The xv6 shell has no `PATH`. A command name is looked up only in the current directory, and all the xv6 programs (`ls`, `cat`, `echo`, ...) live in the root directory `/`. After `cd d` the current directory is `d`, which does not contain the file `ls`, so the program cannot be found. The problem is avoided by giving the full path of the program, for example `/ls ..`, or by returning to the root with `cd /`.
- The most missed feature in daily work would be the command history together with Tab completion, because every command must be typed completely and corrected by retyping. The easiest feature to add would be a simple program search: if the command is not found in the current directory, the shell tries the same name with the prefix `/`, which would work like a `PATH` with a single directory.

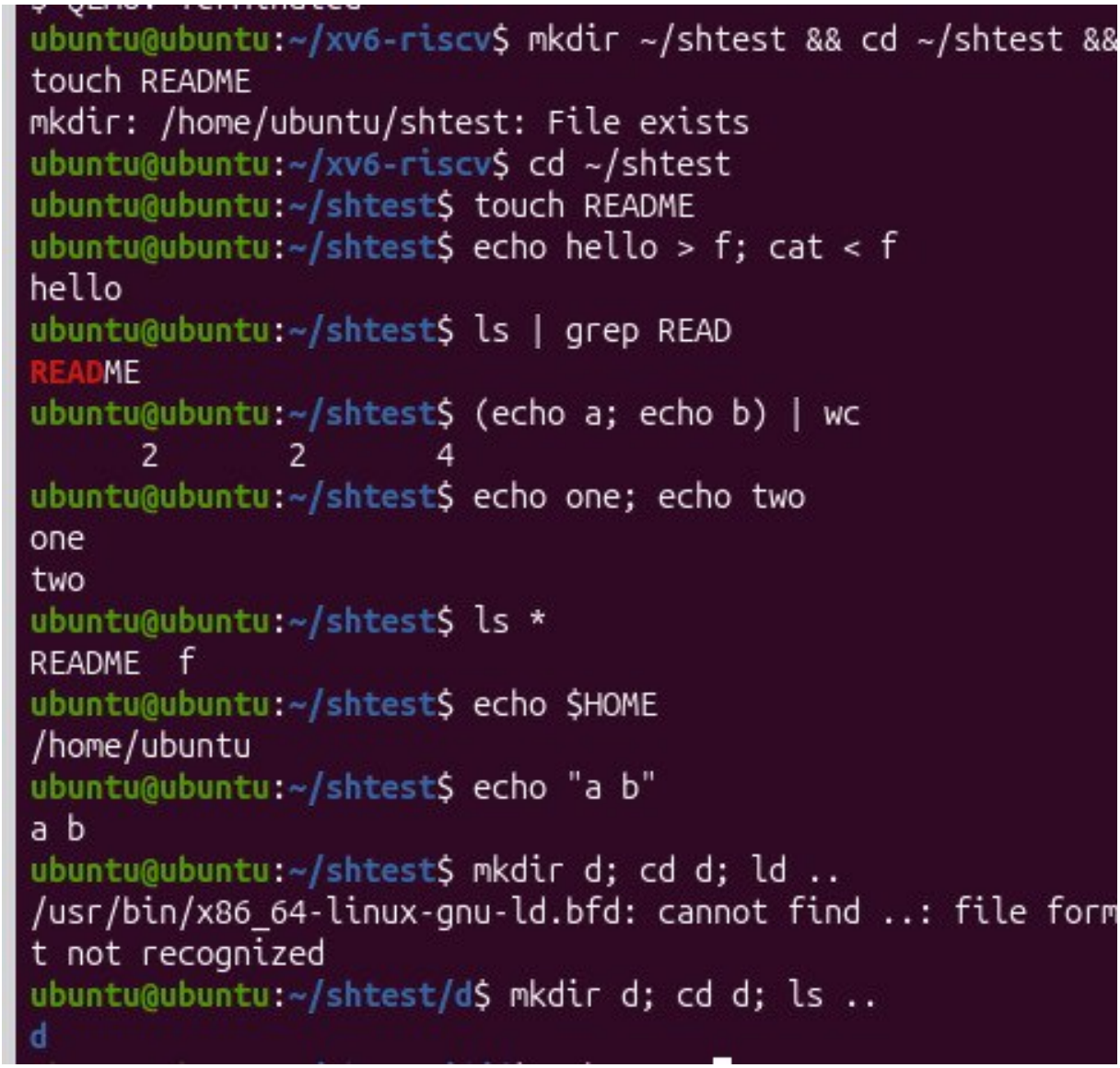


Figure 2.1: The commands run in bash

mkdir ~/shtest && cd ~/shtest && touch README – prepares the test folder with a file named `README`. In Figure 2.1 the command printed `mkdir: File exists`, because the folder had been created earlier; for this reason the chain stopped before `touch`, and the folder was entered and the file was created with separate commands.

echo hello > f ; cat < f – the output of `echo` is written into the file `f`, and then `cat` reads the same file through the input redirection. The printed word `hello` proves that both redirections work.

ls | grep READ – the output of `ls` is sent through a pipe to `grep`, which keeps only the lines containing `READ`. Bash prints `README` and highlights the matched part.

(echo a; echo b) | wc – the parentheses group two commands, and their joint output is counted by `wc`. The result `2 2 4` means 2 lines, 2 words and 4 bytes.

echo one; echo two – the semicolon runs the commands one after the other, so `one` and `two` are printed on separate lines.

ls * – bash expands the asterisk to the names of all the files of the folder, so `ls` receives `README` and `f` and lists them.

echo $HOME and echo "a b" – bash replaces the variable with the home directory `/home/ubuntu` and removes the double quotes, so the printed text is `a b`.

mkdir d ; cd d ; ld .. – the first attempt contained a typing error (`ld` instead of `ls`), and bash answered with the error of the linker program. The command was repeated correctly as `mkdir d; cd d; ls ..`, and since bash found `ls` through the `PATH`, even from inside `d`, it listed the parent folder, which contains the directory `d`.

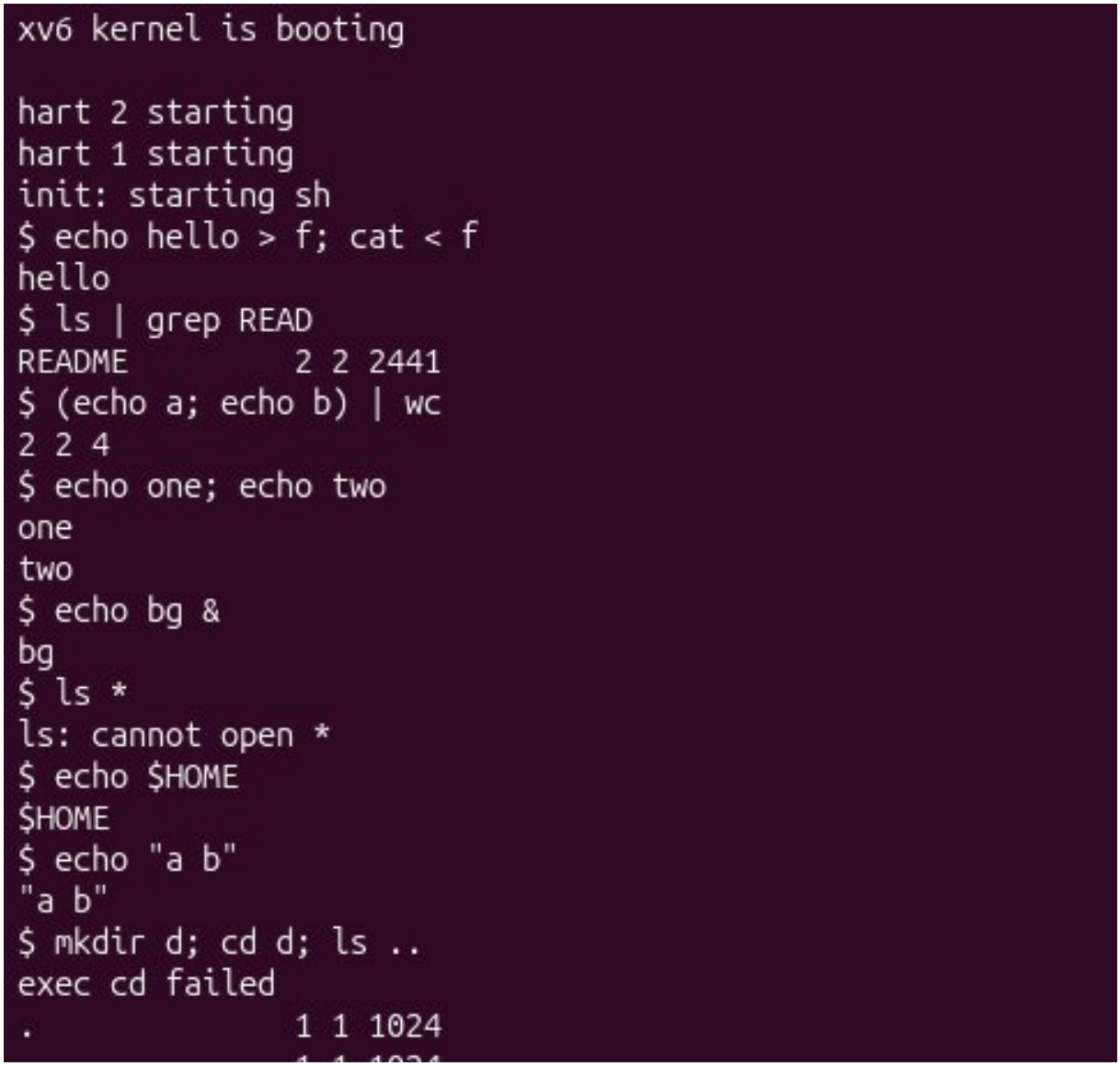


Figure 2.2: The same commands run in xv6

The same commands at the xv6 prompt – Figure 2.2 shows the results in xv6. Redirection, the pipe, the group, the sequence and the background command work in the same way as in bash: `hello`, `README 2 2 2441` (the `ls` of xv6 also prints the file type, the inode number and the size), `2 2 4`, `one`, `two` and `bg`. The differences appear in the last commands: `ls *` prints `ls: cannot open *`, because the asterisk is passed to the program unchanged; `echo $HOME` prints the text `$HOME`, because variables do not exist; and `echo "a b"` prints the quotes together with the text. The last command, `mkdir d; cd d; ls ..`, printed `exec cd failed`, because the shell handles `cd` only when it is the whole line, and when `cd` is part of a list the shell tries to run it as a program, which does not exist. As a result the directory was not changed and `ls ..` listed the root directory.

Command	**Tests**	**bash**	**xv6**	**Why**
`echo hello > f ; cat < f`	redirection	`hello`	`hello`	Both shells open the files and connect them to descriptors 0 and 1.
`ls | grep READ`	pipe	`README`	`README 2 2 2441`	Both create a pipe; the xv6 `ls` prints more fields.
`(echo a; echo b) | wc`	grouping, list	`2 2 4`	`2 2 4`	Parentheses and pipes are supported by both.
`echo one ; echo two`	sequence	`one`, `two`	`one`, `two`	The semicolon is supported by both.
`echo bg &`	background	`bg` (and the job number)	`bg`	Both start the command without waiting.
`ls *`	globbing	`README f`	`ls: cannot open *`	Only bash expands the pattern.
`echo $HOME`	variables	`/home/ubuntu`	`$HOME`	xv6 has no variables.
`echo "a b"`	quotes	`a b`	`"a b"`	xv6 does not remove quotes.
`mkdir d ; cd d ; ls ..`	PATH	`d`	`exec cd failed`, then the listing of the root directory	`cd` inside a list is not run by the xv6 shell, and `ls` is searched only in the current directory.
arrow up, Tab, Ctrl+R	history, completion	work	not available	xv6 reads a whole line without editing support.

Table 1: Comparison of the bash and xv6 shells

The summary of all the results is given in Table 1. The comparison shows that the xv6 shell implements only the basic mechanisms of a Unix shell (processes, pipes, redirection), while the comfort features of bash are additional layers built on top of them.

Part 3: Who expands *? A program that prints its argv

Questions:

- Paste both outputs for args *.c "x y". How many arguments did each program receive?
- So who turns *.c into file names and removes the quotes: the program, the kernel, or the shell?

- What happened to the extra spaces in one two three?

Answers:

- In xv6 the output is `argc = 4` with `argv[0] = [args]`, `argv[1] = [*.c]`, `argv[2] = ["x]` and `argv[3] = [y"]`. In bash the output is `argc = 3` with `argv[0] = [./args]`, `argv[1] = [args_linux.c]` and `argv[2] = [x y]`. The xv6 program received 4 arguments and the Linux program received 3 arguments.
- The shell does both. In bash the pattern `*.c` was replaced with the file name `args_linux.c` and the quotes were removed before the program started, so the program only received the final list of strings. The program does not change its arguments, and the kernel only copies the array of strings into the new process. The xv6 shell does neither of these operations, so the pattern and the quotes reached the program unchanged.
- The extra spaces are only separators between the words. Both shells split the line at the white space and ignore the repeated spaces, so the program receives the three separate arguments `one`, `two` and `three`, and no space is part of any argument.

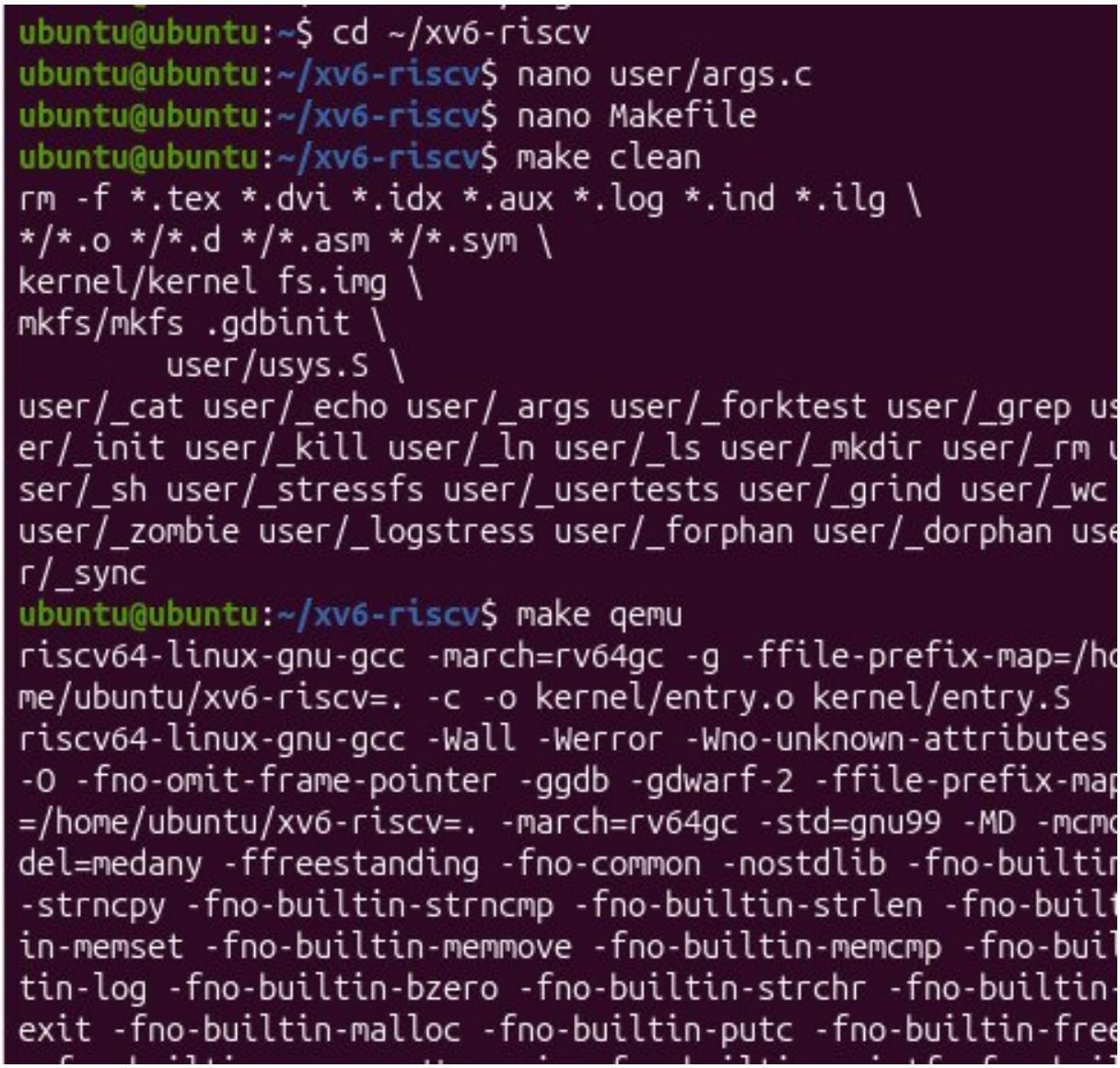


Figure 3.1: The source file user/args.c

nano user/args.c – the program for xv6 includes the two xv6 headers, prints the number of arguments `argc` and then, in a loop, prints every element of the array `argv` between square brackets (Figure 3.1). The brackets make the beginning and the end of every argument visible, so spaces and quotes inside it cannot be missed. The program ends with `exit(0)`, because in xv6 a process must call `exit`.



Figure 3.2: The UPROGS line for args in the Makefile

nano Makefile – the line `$U/_args\` was added to the variable `UPROGS` (Figure 3.2), so that the program is built and copied into the xv6 file system together with `cat`, `echo`, `forktest` and the other programs.



Figure 3.3: args without arguments in xv6

args – the first test in xv6 runs the program without arguments. Figure 3.3 shows `argc = 1` and `argv[0] = [args]`, which proves that the name of the program is always the first element of the array.

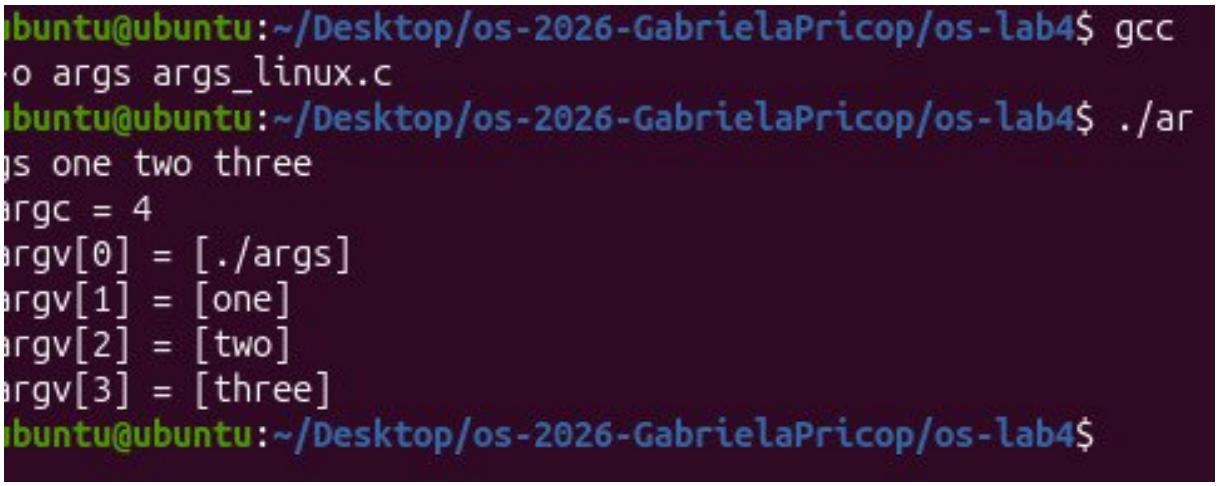


Figure 3.4: args one two three in xv6

args one two three – the xv6 shell splits the line into words, so the program receives `argc = 4`: the program name and the three words (Figure 3.4).

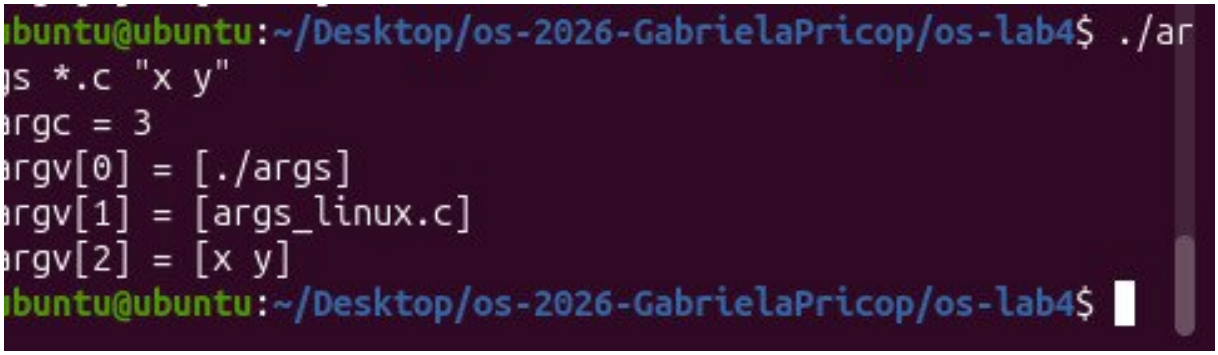


Figure 3.5: args *.c "x y" in xv6

args *.c "x y" – in Figure 3.5 the program received `argc = 4`. The pattern `*.c` was not expanded, and the quoted text was split into two arguments, `"x` and `y"`, each keeping one quotation mark. The xv6 shell therefore treats the quotes as ordinary characters.

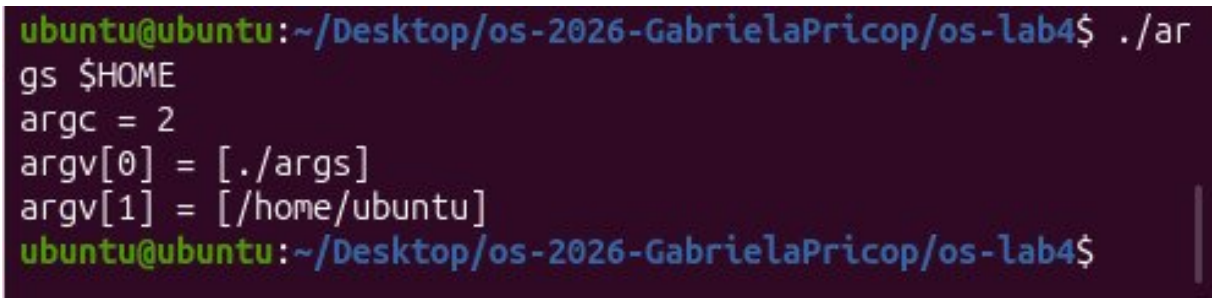


Figure 3.6: args $HOME and args

args $HOME and args 'single quoted' – the variable is not replaced, and the program receives the literal text `[$HOME]`. The single quotes are not processed either, so

```
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab4$ ./ar
gs 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab4$
```

Figure 3.7: single quoted in xv6

`'single quoted'` becomes two arguments, `['single]` and `[quoted']`, and `argc` is 3 (Figure 3.6).

```
  GNU nano 8.7.1                args_linux.c
#include <stdio.h>
int
main(int argc, char *argv[])
{
printf("argc = %d\n", argc);
for(int i = 0; i < argc; i++)
printf("argv[%d] = [%s]\n", i, argv[i]);
return 0;
}
```

Figure 3.8: The source file args_linux.c

nano args_linux.c – the Linux version has the same logic, but it includes `<stdio.h>` and ends with `return 0` instead of `exit(0)` (Figure 3.8).

```
buntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab4$ gcc
o args args_linux.c
buntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab4$ ./ar
gs one two three
rgc = 4
rgv[0] = [./args]
rgv[1] = [one]
rgv[2] = [two]
rgv[3] = [three]
buntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab4$
```

Figure 3.9: Compiling args_linux.c with gcc

gcc -o args args_linux.c – compiles the source file into the executable `args`. The command `ls` in Figure 3.9 lists the new file, which is shown in green as an executable.

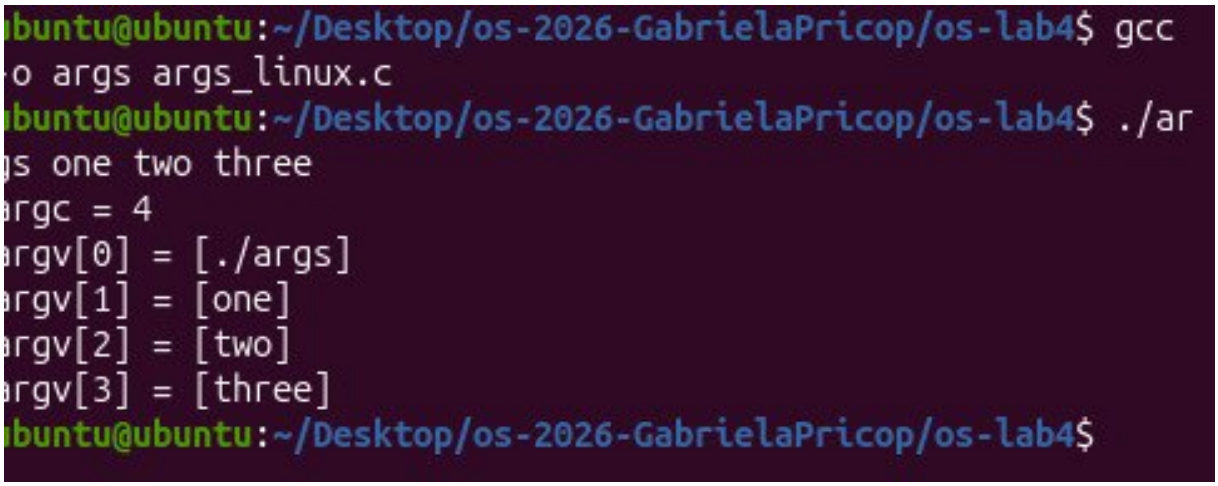


Figure 3.10: ./args one two three in bash

./args one two three – the result in bash is `argc = 4`, the same as in xv6 (Figure 3.10). The only difference is `argv[0]`, which contains `./args`, the path typed by the user, because the program was started with an explicit path.

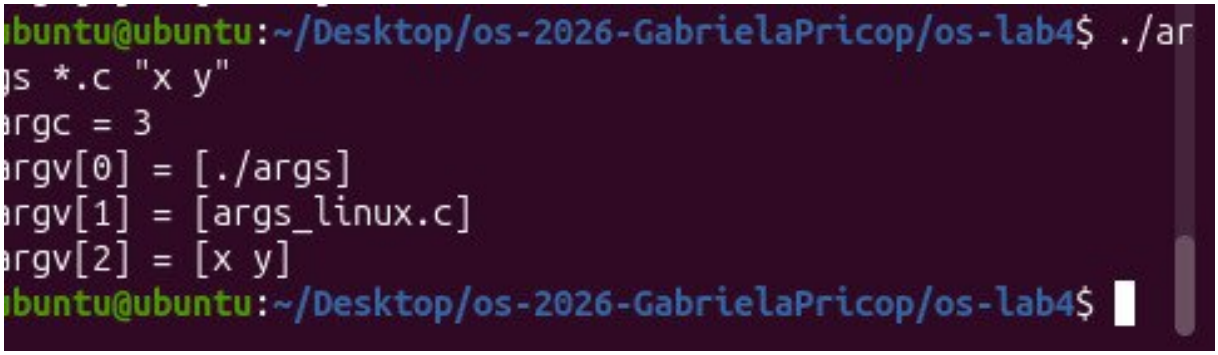


Figure 3.11: ./args *.c "x y" in bash

./args *.c "x y" – in a folder that contains only the file `args_linux.c`, bash replaced the pattern with that name and removed the quotes, so the program received `argc = 3` (Figure 3.11). The text `x y` is a single argument, because the quotes protected the space.

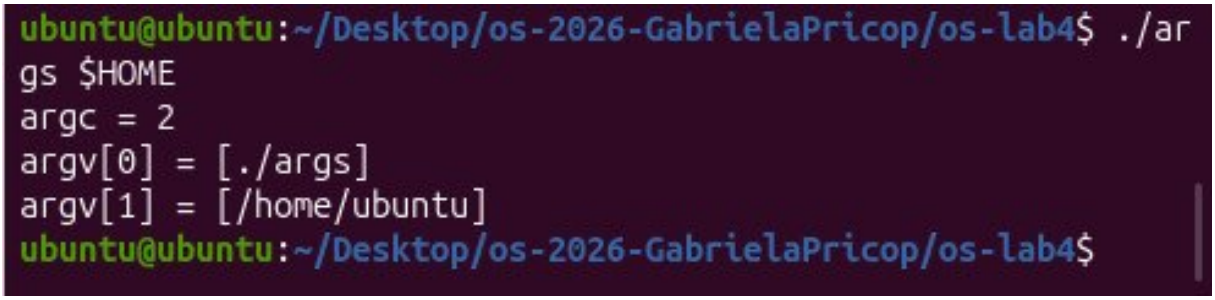


Figure 3.12: ./args $HOME in bash

./args $HOME – bash expanded the variable before starting the program, so `argv[1]` contains `/home/ubuntu` (Figure 3.12).

```
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab4$ ./ar
gs 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
ubuntu@ubuntu:~/Desktop/os-2026-GabrielaPricop/os-lab4$
```

Figure 3.13: ./args 'single quoted' in bash

./args 'single quoted' – the single quotes keep the two words together and are removed, so the program receives one argument, `[single quoted]`, and `argc` is 2 (Figure 3.13).

Command	bash argc	xv6 argc	Reason
`args one two three`	4	4	Both split at spaces.
`args *.c "x y"`	3	4	Only bash expands `*.c` and removes the quotes.
`args $HOME`	2	2	Only bash replaces the value of the variable.
`args 'single quoted'`	2	3	Only bash keeps the quoted text as one argument.

Table 2: Number of arguments received by args in the two shells

Table 2 summarizes the experiment: for the same typed line, the number of arguments depends only on the shell, which proves that the interpretation of the line is done by the shell before the program starts.

Part 4: Read the shell

Questions:

- Name the five kinds of command the xv6 shell knows and the character that produces each.
- Where is cd handled, and why is it not run as a separate program?
- Search sh.c for any code that handles * or $. What does that explain about Part 2?

Answers:

- The five kinds are: `EXEC`, a simple command made of words, which has no special character; `REDIR`, produced by the characters `<`, `>` and `»`; `PIPE`, produced by `|`; `LIST`, produced by `;`; and `BACK`, produced by `&`. The parentheses are used only for grouping, so that a block can be treated as a single command.

- The command `cd` is handled in the main loop of the shell, before a child process is created (the error message `cannot cd` is on line 170 of `user/sh.c`). It cannot be a separate program, because every program runs in a child process, and the current directory belongs to the process: a separate `cd` would change the directory of the child only and then terminate, leaving the shell in the same place. Therefore `chdir` must be called by the shell itself, in the same way as in bash.
- The file `sh.c` contains no code that handles the characters `*` or $ or the quotes. The special characters recognized by the shell are only those of the five kinds of commands and the parentheses. This explains the results of Part 2: `ls *` received a literal asterisk, `echo $HOME` printed the text `$HOME`, and `echo "a b"` kept the quotes.



Figure 4.1: The command kinds defined in sh.c

wc -l user/sh.c – counts the lines of the source file of the shell. The result in Figure 4.1 is 499, which confirms that the whole user interface of xv6 is a program of about 500 lines.

grep -n "#define" user/sh.c – lists the lines that contain macro definitions, together with their line numbers. Figure 4.1 shows the constants `EXEC 1`, `REDIR 2`, `PIPE 3`, `LIST 4` and `BACK 5`, which are the five kinds of commands, and `MAXARGS 10`, which limits the number of words of a simple command.

```
ubuntu@ubuntu:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
ubuntu@ubuntu:~/xv6-riscv$
```

Figure 4.2: Where each kind of command is run

grep -n "case " user/sh.c – shows all the branches of the `switch` statements of the program (Figure 4.2). The lines 75, 83, 93, 101 and 125 are the branches of the function `runcmd`, where each kind of command is executed. The lines 284–294 contain the characters recognized by the function that reads tokens (`|`, `(`, `)`, `;`, `&`, `<` and `>`), and the lines 391–397 handle the redirection symbols, including `+`, which stands for `»`. The last group, from line 469, belongs to the function that terminates the strings of the command tree. No branch exists for `*` or $.

```
ubuntu@ubuntu:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
ubuntu@ubuntu:~/xv6-riscv$
```

Figure 4.3: The code that handles cd

grep -n "cd" user/sh.c | head – searches for the text `cd` and shows the first matches. Figure 4.3 shows the line 170, which prints the message `cannot cd` when the system call

`chdir` fails. This line is in the part of the shell that changes the directory in the shell process itself.

Conclusion

The laboratory work started with the adaptation of the program `sleep` to the current version of xv6, where the system call `sleep()` was renamed to `pause()`, and it showed that a user program depends on the names declared in `user/user.h` and on the line of the Makefile that adds it to `UPROGS`. In the second part, the same commands were executed in bash and in the xv6 shell. The comparison showed that both shells support redirection, pipes, grouping, sequences, background execution and the built-in `cd`, while only bash offers wildcards, variables, quote removal, the `PATH` search and the command history with Tab completion. It also explained why `ls` cannot be found after `cd d` in xv6, where the programs live in the root directory and can be started with a full path such as `/ls`.

The third part used the small program `args`, written for xv6 and for Linux, to show the array of arguments that a program really receives. For the line `args *.c "x y"` the xv6 program received four arguments and the Linux program three, which proved that the expansion of `*`, of variables and the removal of quotes are done by the shell and not by the program or the kernel. In the last part, the source code of `user/sh.c` was read with `wc`, `grep -n "#define"` and `grep -n "case "`. The five kinds of commands (`EXEC`, `REDIR`, `PIPE`, `LIST` and `BACK`) were identified, it was found that the shell contains no code for `*` or $, and it was explained why `cd` must be executed by the shell itself and not by a separate program. Overall, the laboratory work gave a practical understanding of what a shell must do, what bash adds on top of it, and how a program receives its arguments, and all the objectives of the work were achieved.

References

1. *Russ Cox, Frans Kaashoek, Robert Morris. xv6: a simple, Unix-like teaching operating system (RISC-V edition)* `https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf`
2. *xv6-riscv source code repository, MIT PDOS* `https://github.com/mit-pdos/xv6-riscv`
3. *GNU Bash Reference Manual: Shell Expansions* `https://www.gnu.org/software/bash/manual/html_node/Shell-Expansions.html`
4. *Michael Kerrisk. The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press, 2010*